

CourtSplit

Code Documentation

A split-payment court booking platform. A group books a sports court, each player pays only their own share, and the slot is confirmed only once every share has settled. If the group does not complete payment before a deadline, the booking expires and everyone who paid is refunded automatically.

Field	Detail
Author	Ibrahim Ansar
Repository	github.com/novaxertz/courtsplit
Backend	Node.js 18+, Express 4, MongoDB with Mongoose 8
Auth	JWT bearer tokens, bcrypt password hashing
Payments	Stripe (test mode), with an in-memory mock provider
Mobile	React Native via Expo 51
Tests	node:test - 14 passing
Size	24 backend source files, ~1,400 lines

The problem being solved

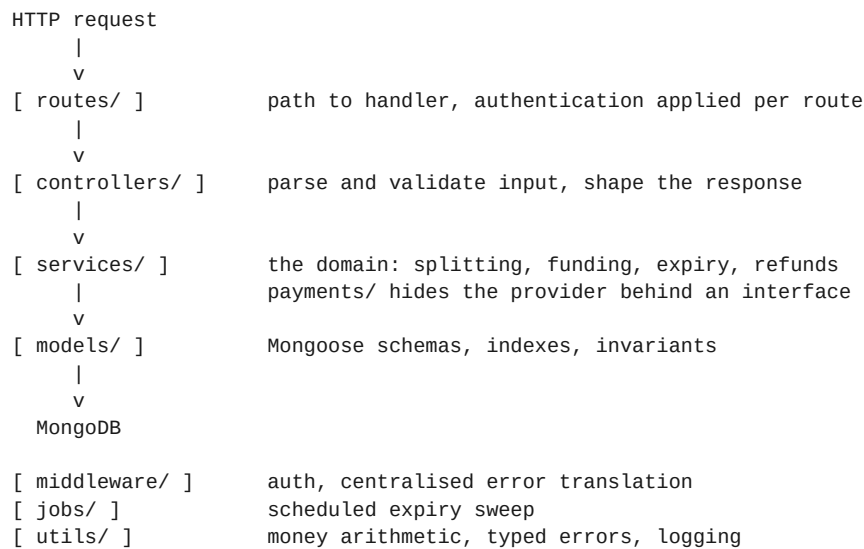
Booking a padel court with friends normally means one person pays the whole amount up front and then chases everyone else for their portion. CourtSplit removes that step by collecting each share independently and treating the booking as unconfirmed until the group is fully funded.

Why it is technically interesting

Most booking applications are CRUD. This one has to keep **money and time correct simultaneously**: a slot may only be sold once even under concurrent requests, a booking is funded by several independent payments so it is neither paid nor unpaid, payment webhooks arrive at least once and must not be double-counted, and an incomplete group must have every collected payment returned.

1. Architecture

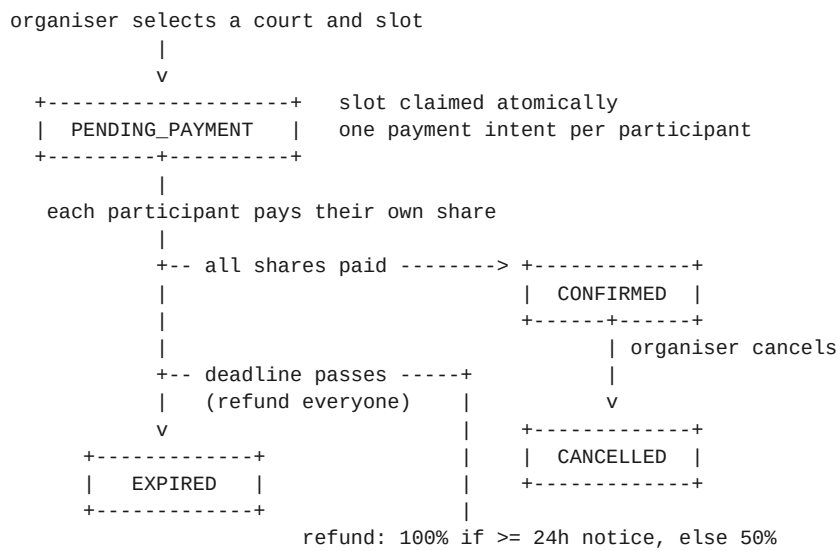
A layered Express application. Each layer has one responsibility and depends only on the layer beneath it, so the domain logic can be read and tested without HTTP or Stripe in the way.



Module map

Path	Responsibility
server.js	Process entry: connect, start listener, graceful shutdown
src/app.js	Express wiring; middleware ordering matters here
src/config/env.js	Environment loading with fail-fast validation
src/config/db.js	Mongo connection and transaction capability probe
src/models/	User, Court, Booking, WebhookEvent
src/controllers/	auth, court, booking, webhook handlers
src/services/bookingService.js	Core domain logic
src/services/payments/	Provider interface, Stripe and mock implementations
src/middleware/	JWT authentication, error handler
src/jobs/expireBookings.js	Periodic expiry sweep
src/utils/money.js	Integer money arithmetic

2. Booking lifecycle



PENDING_PAYMENT and CONFIRMED hold the slot. EXPIRED and CANCELLED release it, making it bookable again.

3. Guaranteeing one booking per slot

Checking whether a slot is free and then inserting is a race. Two requests can both read *free* before either writes, and both succeed. Application-level checks cannot close that window.

The guarantee is therefore delegated to the database through a **partial unique index**:

```
bookingSchema.index({
  { court: 1, slotStart: 1 },
  {
    unique: true,
    partialFilterExpression: {
      status: { $in: ['PENDING_PAYMENT', 'CONFIRMED'] }
    }
  }
});
```

The second concurrent writer receives MongoDB error 11000 (duplicate key), which the service translates into a clean 409 Conflict:

```
try {
  booking = await Booking.create({ ... });
} catch (err) {
  if (err.code === 11000) {
    throw ApiError.conflict('That slot has already been booked');
  }
  throw err;
}
```

Making the index *partial* is what allows a slot to be reused. Only live bookings participate in the uniqueness constraint, so an expired or cancelled booking no longer blocks the time.

4. Handling webhooks safely

4.1 Signature verification

The webhook endpoint is a publicly reachable URL. Without verifying the provider's signature, anyone could POST a forged `payment_intent.succeeded` and confirm a booking they never paid for.

Verification hashes the exact bytes the provider sent, so the raw body must be preserved. That is why the webhook route is mounted with a raw parser *before* the global JSON parser in `app.js` - re-serialising parsed JSON would not reproduce the original bytes and every signature would fail.

```
// src/app.js - ordering is deliberate
app.post('/webhooks/payments',
  express.raw({ type: 'application/json' }),
  webhookController.handle);

app.use(express.json()); // everything after this point gets parsed JSON
```

4.2 Idempotency

Payment providers retry delivery until they receive a 2xx response, so the same event routinely arrives more than once. Acting on each delivery would mark a share paid twice and could confirm or refund incorrectly.

Each event id is inserted into a collection with a unique index before any work happens. The first delivery inserts and proceeds; every replay collides and returns early.

```
try {  
  await WebhookEvent.create({ eventId: event.id, type: event.type });  
} catch (err) {  
  if (err.code === 11000) {  
    return res.json({ received: true, duplicate: true });  
  }  
  throw err;  
}
```

This turns replay protection into a single indexed insert rather than a lock or a read-then-write check.

5. State transitions as conditional writes

Even with webhook de-duplication, two different events can act on one booking at the same moment - for example the final share settling while the expiry sweep is running. A read-modify-write sequence would allow one to overwrite the other.

Every transition is therefore expressed as a single conditional update that matches on the state it expects to find:

```
const updated = await Booking.findOneAndUpdate(
  {
    _id: bookingId,
    status: 'PENDING_PAYMENT',
    shares: { $elemMatch: { _id: shareId, status: 'pending' } }
  },
  {
    $set: {
      'shares.$.status': 'paid',
      'shares.$.paidAt': new Date()
    }
  },
  { new: true }
);
```

If the share has already settled, or the booking is no longer pending, the filter matches nothing, no document changes, and the operation is a safe no-op. The same pattern guards confirmation, expiry and cancellation.

Confirmation is guarded a second time so a concurrent expiry cannot be silently overwritten:

```
if (updated.shares.every((s) => s.status === 'paid')) {
  const confirmed = await Booking.findOneAndUpdate(
    { _id: updated._id, status: 'PENDING_PAYMENT' }, // still pending?
    { $set: { status: 'CONFIRMED', confirmedAt: new Date() } },
    { new: true }
  );
}
```

6. Money handling

6.1 Integers, never floats

All amounts are stored in the smallest currency unit - fils, where 1 AED = 100. Floating point is unsuitable for money: $0.1 + 0.2$ does not equal 0.3 in binary floating point, and those errors accumulate across splits and refunds.

6.2 Splitting without losing value

A naive division drops the remainder, quietly short-changing the venue. The split distributes it one unit at a time:

```
function splitAmount(total, n) {
  const base = Math.floor(total / n);
  const remainder = total % n;
  return Array.from({ length: n },
    (_, i) => base + (i < remainder ? 1 : 0));
}
```

```
24000 across 4 -> 6000 + 6000 + 6000 + 6000
10000 across 3 -> 3334 + 3333 + 3333 (sums back to 10000)
```

The invariant that the parts sum back to the total is asserted across hundreds of total and participant combinations in the test suite, not just for a couple of hand-picked examples.

7. Payment provider abstraction

The booking logic never imports Stripe. It depends on a small interface that both implementations satisfy, so the domain code is unaware of which processor fulfilled a request.

Method		Purpose
createPaymentIntent()		Create a charge intent for one participant's share
refund()		Refund all or part of a settled share
constructEvent()		Verify and parse an incoming webhook
Provider	When used	Behaviour
mock	Default; no keys required	In-memory intents, so the full flow runs immediately after clone
stripe	PAYMENT_PROVIDER=stripe	Real Stripe test API with idempotency keys and signature checks

Shipping a working mock matters for a public repository: anyone can clone the project and exercise booking, funding, expiry and refunds without creating a Stripe account.

Stripe requests also carry idempotency keys, so a retried create reuses the original intent instead of charging a second time:

```
idempotencyKey: `booking_${booking.id}_share_${share._id}`
```

8. Expiry sweep

A booking that is never fully funded must not hold a slot forever. A periodic sweep finds bookings past their funding deadline, claims each one with a conditional transition, and refunds whoever had paid.

```
const claimed = await Booking.findOneAndUpdate(
  { _id: candidate._id, status: 'PENDING_PAYMENT' },
  { $set: { status: 'EXPIRED', expiredAt: now } },
  { new: true }
);
if (!claimed) continue; // another worker claimed it first
await refundShares(claimed, 100);
```

Because the claim is conditional, running the sweep more often than necessary, or on several instances simultaneously, cannot refund anyone twice.

9. Authentication

- Passwords hashed with bcrypt at 10 rounds; the hash is stripped from every JSON response by a schema transform rather than by remembering to delete it at each call site
- Stateless JWT bearer tokens carrying user id, email and role
- Login returns an identical error whether the email is unknown or the password is wrong, so the endpoint cannot be used to enumerate which addresses are registered
- Configuration fails fast at startup if JWT_SECRET or MONGO_URI is missing, rather than failing later at request time

10. API reference

Method	Endpoint	Auth	Purpose
POST	/auth/register	-	Create an account, returns a JWT
POST	/auth/login	-	Authenticate, returns a JWT
GET	/auth/me	JWT	Current user
GET	/courts	-	List courts; filter by sport, location, price
POST	/courts	JWT	Add a court
GET	/courts/:id/availability	-	Slots already taken in a range
POST	/bookings	JWT	Create booking and per-player intents
GET	/bookings	JWT	Bookings you organise or belong to
GET	/bookings/:id	JWT	One booking with share breakdown
POST	/bookings/:id/cancel	JWT	Cancel and refund per policy
POST	/webhooks/payments	Signature	Provider callback
GET	/health	-	Liveness check

Creating a booking returns the share breakdown so each player can be shown exactly what they owe:

```
{
  "booking": {
    "status": "PENDING_PAYMENT",
    "totalAmount": 24000,
    "fundingDeadline": "2026-07-29T15:30:00.000Z",
    "shares": [
      { "email": "you@example.com", "amount": 8000, "status": "pending" },
      { "email": "sam@example.com", "amount": 8000, "status": "pending" },
      { "email": "alex@example.com", "amount": 8000, "status": "pending" }
    ]
  }
}
```

11. Testing

Tests target the logic where mistakes are expensive and silent - money arithmetic and payment provider behaviour - rather than chasing coverage of straightforward handlers.

```
money.test.js          9 tests
  splits evenly when the total divides exactly
  distributes the remainder instead of losing it
  parts always sum back to the total across many combinations
  no share is ever negative
  shares differ by at most one minor unit
  rejects invalid inputs rather than returning nonsense
  percentageOf rounds to the nearest whole unit
  percentageOf rejects out-of-range percentages
  format renders minor units as a readable amount
```

```
paymentProvider.test.js 5 tests
  creates an intent carrying the booking metadata
  confirming an intent moves it to succeeded
  refund returns the requested amount
  unknown intents are rejected rather than silently ignored
  constructEvent parses a provider payload
```

14 passing

Extracting money arithmetic into `utils/money.js` keeps it free of database and network dependencies, so these assertions run in milliseconds without a live MongoDB.

12. Scope and limitations

Built as a portfolio project. The following are deliberate omissions rather than oversights.

Not implemented	What production would require
Payouts to venues	Stripe Connect onboarding and transfer scheduling
Native card entry in the app	Stripe payment sheet integration on iOS and Android
Dispute and admin console	Resolution workflow with evidence handling
Durable scheduling	The expiry sweep is an in-process interval; a restart can skip a window
Email invitations	Participants must already hold an account; matching is by email lookup
Rate limiting	Throttling on auth and booking creation
Ledger	Append-only double-entry records for full financial auditability

13. Running the project

13.1 Backend

```
cd backend
npm install
cp .env.example .env      # fill in MONGO_URI and JWT_SECRET
npm run seed              # optional demo users and courts
npm run dev               # -> CourtSplit API listening on :4000
```

No payment keys are needed. `PAYMENT_PROVIDER` defaults to `mock`, which exercises the entire booking flow in memory.

13.2 Mobile

```
cd mobile
npm install
npm start                # scan the QR with Expo Go, or press w for web
```

A physical device cannot reach `localhost`. Set `API_URL` in `mobile/config.js` to the machine's LAN IP when testing on hardware.

13.3 Tests

```
cd backend
npm test
```

Appendix - Configuration

Variable	Default	Purpose
<code>MONGO_URI</code>	required	MongoDB connection string
<code>JWT_SECRET</code>	required	Token signing secret
<code>PORT</code>	4000	HTTP port
<code>PAYMENT_PROVIDER</code>	mock	mock or stripe
<code>STRIPE_SECRET_KEY</code>	-	Required when provider is stripe

STRIPE_WEBHOOK_SECRET	-	Signature verification secret
FUNDING_WINDOW_MINUTES	60	Time a group has to fully fund
FREE_CANCELLATION_HOURS	24	Notice needed for a full refund
LATE_CANCELLATION_REFUND_PERCENTAGE	50	Refund inside the free window

Secrets are read from the environment and never committed. The tracked `.env.example` documents which variables exist using placeholders only.